

One way to print the individual items in the list would be to split the list into multiple arguments by passing each item's index to the function individually, resulting in code that is harder to read:

```
>>> # An example of less readable code:
>>> args = ['cat', 'dog', 'moose']
>>> print(args[0], args[1], args[2])
cat dog moose
```

There's an easier way to pass these items to `print()`. You can use the `*` syntax to interpret the items in a list (or any other iterable data type) as individual positional arguments. Enter the following example into the interactive shell.

```
>>> args = ['cat', 'dog', 'moose']
>>> print(*args)
cat dog moose
```

The `*` syntax allows you pass the list items to a function individually, no matter how many items are in the list.

You can use the `**` syntax to pass mapping data types (such as dictionaries) as individual *keyword arguments*. Keyword arguments are preceded by a parameter name and equal sign. For example, the `print()` function has a `sep` keyword argument that specifies a string to put in between the arguments it displays. It's set to a single space string `' '` by default. You can assign a keyword argument to a different value using either an assignment statement or the `**` syntax. To see how this works, enter the following into the interactive shell:

```
>>> print('cat', 'dog', 'moose', sep='-')
cat-dog-moose
>>> kwargsForPrint = {'sep': '-'}
>>> print('cat', 'dog', 'moose', **kwargsForPrint)
cat-dog-moose
```

Notice that these instructions produce identical output. In the example, we used only one line of code to set up the `kwargsForPrint` dictionary. But for more complex cases, you might need more code to set up a dictionary of keyword arguments. The `**` syntax allows you to create a custom dictionary